Agentic 티켓 시스템 구축 방안 보고서

작성일: 2026년 3월 20일 **대상**: Java/Spring Boot 개발자 **목표**: 티켓 발행 시 자동 브랜치 생성 + AI 에이전트 코드 생성

---

1. 요구사항 요약

항목	내용
티켓 발행	Jira-like 티켓 생성
브랜치 자동 생성	베이스 브랜치에서 티켓 ID 기반 브랜치 자동 생성
AI 계획 수립	티켓 설명 기반으로 구현 계획 자동 생성
코드 자동 생성	에이전트가 자율적으로 코드 작성 및 PR 생성
개발자 배경	Java/Spring Boot 전문, Node.js/Nuxt 비경험

---

2. 기술 스택 선택지 비교

2-A. GitHub Issues (강력 추천 ★★★★★)

GitHub Issues를 티켓 소스로 사용하는 방식. 별도 UI 개발 없이 GitHub 생태계를 그대로 활용.

장점

- 별도 UI 서비스 불필요 — 개발 비용 Zero
- GitHub Actions과 네이티브 통합 (webhook 자동)
- `anthropics/claude-code-action@v1` 공식 액션 즉시 사용 가능
- 프로젝트의 `CLAUDE.md`, `CONVENTIONS.md` 를 Claude가 자동으로 읽어 DDD 규칙 준수
- Node.js 로컬 설치 불필요 (GitHub 런너 안에서만 동작)

단점

- 스프린트, 스토리 포인트 등 고급 기능은 GitHub Projects를 별도로 설정해야 함
- 상태 머신(진행중/검토중 등)이 단순 open/closed로만 구분됨

비용: 무료 (공개 저장소) / 월 $4~ (비공개)

2-B. Linear (추천 ★★★★☆)

전문 이슈 트래커. GitHub 연동으로 자동 브랜치명 생성 지원.

장점

- 스프린트, 로드맵, 우선순위, 사이클 등 풍부한 PM 기능
- 자동 브랜치명 형식: `eng-123-feature-name`
- PR 열리면 Linear 티켓과 자동 연동, 머지 시 티켓 자동 완료

단점

- 브랜치 자동 생성은 UI 단축키(Cmd+Shift+.) 제공이며, 완전 자동화는 webhook + GitHub Actions 추가 구성 필요

- 유료 서비스 (팀 플랜 $8/유저/월~)
- GitHub Issues보다 통합 레이어가 복잡

2-C. Spring Boot + Thymeleaf 직접 구축 (비추천 ★★☆☆☆)

커스텀 티켓 트래커를 Spring Boot로 직접 개발.

장점

- Java만으로 완성 가능, 기술 스택 통일
- DDD 구조로 완전 커스텀 가능 (`domain/ticket` , `application/ticket` 등)
- Anthropic Java SDK 직접 연동 가능

단점

- 수 주~수 개월의 개발 공수 (티켓 CRUD, 상태 관리, UI 등)
- 결국 GitHub Issues가 무료로 제공하는 기능을 재구현하는 셈
- Thymeleaf로는 실시간 보드(드래그앤드롭 등) 구현이 어려움

2-D. Spring Boot + Nuxt (비추천 ★☆☆☆☆)

사용자가 Node.js를 모른다고 명시 → 제외.

---

3. AI 에이전트 도구 비교

3-A. Claude Code GitHub Action (최강 추천 ★★★★★)

Anthropic 공식 제공. GitHub Actions에서 Claude Code 에이전트 루프 전체를 실행.

```
공식 액션: anthropics/claude-code-action@v1
```

동작 원리

```
이슈 생성 (ai-implement 라벨 or @claude 포함)
    ↓
GitHub Actions 트리거
    ↓
Claude가 CLAUDE.md + CONVENTIONS.md 읽기
    ↓
티켓 설명 분석 → 구현 계획 수립
    ↓
코드 작성 (DDD 레이어 순서대로)
    ↓
./gradlew build + test 실행
    ↓
Draft PR 자동 생성 (이슈 #번호 링크 포함)
    ↓
개발자 검토 → @claude 코멘트로 수정 요청 → 재작업
    ↓
승인 + 머지
```

핵심 옵션

옵션	설명
`--max-turns 20`	최대 반복 횟수 제한 (비용 제어)
`--allowedTools`	사용 가능한 도구 화이트리스트
`--model claude-sonnet-4-6`	모델 지정
`prompt`	구현 지침 전달 (CLAUDE.md 참조 포함)

3-B. Claude Agent SDK (고급 커스터마이징 ★★★☆☆)

Python/TypeScript SDK로 에이전트 루프를 프로그래밍적으로 제어.

- Python: `pip install claude-agent-sdk`
- TypeScript: `npm install @anthropic-ai/claude-agent-sdk`
- **Java SDK 없음** (2026년 3월 기준) → Spring Boot에서 사용 시 Python 사이드카 or `ProcessBuilder` 필요

3-C. Anthropic Java Client SDK (단순 API 호출 ★★☆☆☆)

Maven: `com.anthropic:anthropic-sdk-java`

Messages API만 제공. 에이전트 루프(도구 실행 → 결과 피드백 → 반복)를 직접 구현해야 함. 코드 생성 자동화에는 과도한 구현 부담.

3-D. Sweep AI (비추천 ★★☆☆☆)

과거 오픈소스 GitHub 봇이었으나 2025년 초 유료 전환. 신규 프로젝트 비추천.

3-E. GitHub Copilot Workspace (미래 고려 ★★★☆☆)

GitHub 이슈에서 Copilot이 계획 및 코드 제안. 2026년 기준 Enterprise 전용 프리뷰. 프로그래밍 API 없음 — 자동화 파이프라인 적용 불가.

---

4. 최종 추천 아키텍처

조합: GitHub Issues + `claude-code-action@v1`

이유

- Java 개발자가 Node.js 없이 즉시 적용 가능
- 이 프로젝트의 `CLAUDE.md`가 이미 DDD 규칙을 정의 → Claude가 자동 준수
- `bash-guard.sh` 등 기존 보안 훅과 별도로, `--allowedTools`로 Actions 런너에서의 권한도 제어
- PR 코멘트에서 `@claude`로 추가 수정 요청 가능 (대화형 개발)

---

5. GitHub Actions 워크플로 (완성본)

`.github/workflows/claude-implement.yml`

```yaml
name: Claude Auto-Implement
```

```yaml
on:
  issues:
    types: [opened, labeled]
  issue_comment:
    types: [created]

jobs:
  claude-implement:
    # 이슈: 'ai-implement' 라벨 또는 본문에 @claude 포함
    # 코멘트: @claude 포함 시
    if: |
      (github.event_name == 'issues' && (
        contains(github.event.issue.labels.*.name, 'ai-implement') ||
        contains(github.event.issue.body, '@claude')
      )) ||
      (github.event_name == 'issue_comment' &&
        contains(github.event.comment.body, '@claude'))
    runs-on: ubuntu-latest
    timeout-minutes: 30
    permissions:
      contents: write
      pull-requests: write
      issues: write
      id-token: write

    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'

      - name: Create feature branch
        id: branch
        if: github.event_name == 'issues'
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          ISSUE_NUM="${{ github.event.issue.number }}"
          TITLE=$(echo "${{ github.event.issue.title }}" ₩
            | tr '[:upper:]' '[:lower:]' ₩
            | sed 's/[^a-z0-9]/-/g' ₩
            | sed 's/--*/-/g' ₩
            | cut -c1-40)
          BRANCH="feat/issue-${ISSUE_NUM}-${TITLE}"
          echo "branch_name=${BRANCH}" >> $GITHUB_OUTPUT
          BASE_SHA=$(gh api repos/${{ github.repository }}/git/refs/heads/main ₩
```

```
            --jq '.object.sha')
          gh api repos/${{ github.repository }}/git/refs ₩
            -f ref="refs/heads/${BRANCH}" ₩
            -f sha="${BASE_SHA}" ₩
            || echo "Branch may already exist, continuing"

      - name: Run Claude Code Agent
        uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}
          prompt: |
            이 프로젝트는 Java 21 + Spring Boot 3.4.4 기반 DDD 멀티모듈 프로젝트입니다.

            GitHub Issue #${{ github.event.issue.number }}: ${{ github.event.issue.title }}

            설명:
            ${{ github.event.issue.body }}

            구현 지침:
            1. CLAUDE.md와 docs/conventions/CONVENTIONS.md를 먼저 읽을 것
            2. 브랜치 ${{ steps.branch.outputs.branch_name }}에서 작업할 것
            3. DDD 레이어 순서대로 구현: domain → application → infrastructure → api
            4. 도메인 로직에 대한 단위 테스트 작성 (TDD)
            5. ./gradlew :apps:chat:chat-server:build 로 컴파일 확인
            6. ./gradlew :apps:chat:chat-server:test 로 테스트 확인
            7. Draft PR 생성 (Closes #${{ github.event.issue.number }} 포함)
          claude_args: |
            --max-turns 20
            --allowedTools "Read,Edit,Write,Bash(git *),Bash(./gradlew *),Bash(gh pr
create*),Glob,Grep"
            --model claude-sonnet-4-6
```

---

6. 브랜치 명명 규칙

소스	형식	예시
GitHub Issues	feat/issue-{번호}-{제목-kebab}	feat/issue-42-add-mfa-rate-limiting
Linear	eng-{번호}/{제목-kebab}	eng-42/add-mfa-rate-limiting

---

7. 검토 및 승인 플로우

```
이슈 생성 (ai-implement 라벨)
    ↓
Actions: 브랜치 자동 생성
    ↓
Claude: 코드 작성 + 테스트 실행 (최대 30분)
    ↓
Claude: Draft PR 생성 (구현 이유 설명 포함)
```

```
    ↓
개발자: PR 코드 리뷰
    ↓
수정 필요 → PR 코멘트에 "@claude 이 부분 수정해줘"
    ↓
Claude: 재작업 (issue_comment 트리거)
    ↓
개발자: Approve → main 머지
```

보호 규칙: `main` 브랜치에 "1명 이상 승인 필수" 규칙 설정 → Claude가 자기 PR을 스스로 머지 불가

---

8. 비용 예측

항목	비용
GitHub Actions 실행 (10분/이슈)	~$0.10/이슈
Claude Sonnet API (50k~200k 토큰/이슈)	~~$0.15~~$0.60/이슈
합계	**$0.25~$0.70/이슈**

---

9. 보안 고려사항

위협	대응
Claude가 위험 명령 실행	`--allowedTools`로 허용 명령 화이트리스트
Actions 런너 탈출	`timeout-minutes: 30` + 별도 Actions 권한 최소화
비밀 키 노출	`ANTHROPIC_API_KEY`는 repository secrets 저장
Claude가 main 직접 push	브랜치 보호 규칙 + Draft PR로만 생성
악의적 이슈로 프롬프트 인젝션	라벨 기반 트리거 (`ai-implement`)로 신뢰된 레이블만 허용

---

10. 단계별 적용 로드맵

단계	내용	예상 공수
1단계	GitHub Issues 라벨 체계 정의 + 워크플로 YAML 추가	2시간
2단계	`ANTHROPIC_API_KEY` secrets 등록 + Claude GitHub App 설치	30분
3단계	간단한 이슈로 파일럿 테스트 (e.g., 작은 버그 수정)	1시간
4단계	`--allowedTools` 세밀 조정 + PR 템플릿 정비	1시간
5단계	(선택) Linear 연동으로 PM 기능 추가	4시간

결론

Java 개발자가 Node.js 없이 가장 빠르게 구현할 수 있는 조합은:

> ***GitHub Issues** (티켓) + `anthropics/claude-code-action@v1` (AI 에이전트)*

이 프로젝트에는 이미 `CLAUDE.md` 와 `CONVENTIONS.md` 가 있어 Claude가 DDD 규칙을 자동으로 준수합니다. 워크플로 YAML 하나를 추가하는 것만으로 전체 파이프라인이 동작합니다.

본 보고서는 Claude Code (claude-sonnet-4-6)가 생성하였습니다.